

HOTEL MANAGEMENT SYSTEM PROJECT REPORT

Name: I Adarsh

Registration no.: 240905294

Section: CSE A

Roll no.: 36

Date: 6 April 2026

Hotel Management System

Mini-Project for OOSDL Lab

1. Introduction

The hospitality industry depends heavily on accurate room allocation, timely check-in and check-out processes, and clear communication between staff and guests. In small and medium-scale hotels, these operations are often handled manually or through fragmented tools, which can lead to overbooking, inconsistent records, and delays in service. As hotels serve multiple guest requests daily, the need for a unified software-based management system becomes essential.

This project presents a Hotel Management and Booking System, a desktop application built using JavaFX, Maven, and SQLite. The system is designed to support two primary user roles: admin and guest. The admin role focuses on operational oversight, while the guest role handles booking-related interactions. By combining user authentication, role-based navigation, and persistent database storage, the application aims to digitize core hotel workflows in a reliable and user-friendly way.

A major feature of this project is automatic database setup during application startup. The system initializes required tables, seeds room inventory, and ensures a default admin account exists. This approach simplifies setup and allows the application to run with minimal manual configuration. In addition, the system includes booking enhancements such as date-based stay calculation, automatic pricing, and room upgrade support.

From a software engineering perspective, the project demonstrates practical use of layered architecture. Controllers manage UI events, DAO classes handle data access logic, model classes represent domain entities, and a database utility ensures stable connectivity. This separation improves code maintainability and supports future enhancements.

Overall, the project addresses a realistic business problem through a structured implementation that balances usability, functionality, and modular design. It serves both as a practical hotel utility prototype and as a learning exercise in desktop application development with database integration.

2. Objectives

The primary goal of this project is to design and implement a functional hotel management application that supports everyday booking and administration activities. The specific objectives are as follows:

- Develop a role-based desktop system that differentiates access and actions for admin and guest users.
- Implement secure and progressive user access flow with support for guest registration and login-based routing.
- Automate core booking operations, including room selection by category, date-based stay duration, and total cost calculation.
- Enable room status management so that bookings, upgrades, and checkouts correctly update room availability.
- Provide administrative control features, such as viewing active bookings and modifying room type and pricing tiers.
- Maintain persistent hotel data using SQLite, ensuring booking and user records are saved across sessions.
- Use a clean software structure through model, DAO, controller, and utility layers for better maintainability.
- Deliver a practical and user-friendly interface using JavaFX views with consistent styling and theme support.

By achieving these objectives, the project aims to produce a dependable baseline system for hotel operations while also establishing a foundation for advanced future features.

3. Methodology

The project was developed using a modular and incremental methodology, where core workflows were implemented first and then expanded with operational enhancements such as upgrades, admin controls, and startup database initialization.

3.1 Technology Stack

The following technologies were used:

- **Java 17** for core application logic.
- **JavaFX 21** (javafx-controls and javafx-fxml) for the desktop user interface.
- **Maven** for dependency management and build lifecycle.
- **SQLite** (sqlite-jdbc) for lightweight local database storage.

This stack was selected because it is stable, easy to configure for academic projects, and suitable for standalone desktop deployment.

3.2 System Architecture

The project follows a layered architecture:

- **Controllers layer:** Handles UI actions and scene interactions.
- **DAO layer:** Encapsulates SQL operations for users, rooms, and bookings.
- **Models layer:** Defines entity classes such as User, Room, and Booking.
- **Database utility layer:** Manages connection setup and DB initialization behavior.
- **UI resources:** FXML files define views, while CSS files control styling and theme behavior.

This structure separates responsibilities and avoids tightly coupling UI and database logic.

3.3 Database Initialization Strategy

A startup initialization class ensures the application can run without manual database preparation. On launch, the system:

- Creates or verifies required tables (User, Room, Booking).
- Ensures booking schema fields for check-in and check-out dates are present.
- Seeds room inventory if missing or inconsistent.
- Ensures a default admin account exists.

Room seeding follows predefined business rules:

- Floors from ground (0) to 10.
- 15 rooms per floor.
- 165 rooms total.
- Per-floor room distribution: 4 Single, 4 Double, 4 Deluxe, 2 Suite, 1 Presidential Suite.

This deterministic setup provides predictable system behavior for testing and demonstration.

3.4 Authentication and Role Routing

The login process is designed as a progressive flow:

1. User enters username.
2. If the user exists, the system requests a password and performs login.
3. If a user does not exist, the system allows guest registration.

4. After successful authentication, user is routed by role:
 - Admin -> Admin dashboard
 - Guest -> Guest dashboard

This method improves usability for new users while preserving role-based control.

3.5 Booking Workflow Implementation

The guest workflow supports complete room booking operations:

1. Select room type (tier-based preference).
2. Select check-in and check-out dates.
3. The system computes the number of nights.
4. The system calculates total cost based on nightly room price.
5. The system allocates a random available room from the selected category.
6. Booking record is stored and room availability is updated.

Post-booking actions include:

- **Room upgrade:** guests may move to a higher tier and pay the difference.
- **Checkout:** booking is removed and room returns to available status.
- These operations ensure lifecycle consistency from reservation to release.

3.6 Admin Workflow Implementation

The admin interface focuses on supervision and configuration:

- View currently booked rooms with guest name, booking dates, price per night, and total amount.
- Modify room type labels and associated prices globally.

This design allows administrators to adapt room pricing policies and monitor occupancy in one place.

3.7 User Interface and Theme Handling

The UI is built using FXML-based views and shared CSS stylesheets. A theme manager applies:

- Base stylesheet for common components.
- Light or dark theme stylesheet based on operating system preference.

This improves visual consistency and provides a better user experience across environments.

4. Results and Discussion

The implemented system successfully demonstrates end-to-end hotel booking and management functionality in a desktop environment.

4.1 Functional Results

The following outcomes were observed during testing and execution:

1. Application startup and initialization: The database file is created automatically, required tables are verified, and initial records are prepared.
2. Authentication and role handling: Existing users can log in; new users can register as guests; dashboards load according to role.
3. Guest booking execution: Guests can select room category and date range, and the system correctly computes nights and total cost.
4. Room allocation and state updates: Rooms are assigned from available stock, and occupancy status changes are reflected in data records.
5. Upgrade and checkout flow: Guests can upgrade to a higher tier and complete checkout, after which availability is restored.
6. Admin operations: Admin can review booking records and adjust room type names or pricing definitions.

Collectively, these results confirm that the project meets its intended operational scope.

4.2 Discussion of Strengths

Several strengths emerged from the final implementation:

- **Practical workflow coverage:** The project handles both guest-side and admin-side operations, making it more realistic than a single-role prototype.
- **Automatic setup:** Startup initialization and seeding reduce setup friction and improve reproducibility.
- **Modular code organization:** DAO and controller separation supports easier maintenance and extension.
- **Consistent pricing logic:** Date-based cost calculation and tier management provide coherent booking behavior.
- **Simple deployment model:** SQLite integration avoids external server dependencies, which is ideal for local desktop use.

4.3 Limitations and Risks

Despite successful core implementation, some limitations remain:

- Password storage is plain text, which is not production-safe.
- No automated tests are currently implemented, so regression risk is higher after changes.
- Single-session assumptions in controller state limit multi-user concurrency scenarios.
- Basic validation and exception feedback can be strengthened for edge-case handling.

These limitations are acceptable for an academic prototype but should be addressed for real-world deployment.

4.4 Future Improvements

Potential upgrades include:

- Password hashing and stronger authentication safeguards.
- Unit and integration testing for DAO and controller layers.
- Better booking conflict checks and validation messages.
- Billing receipts and report export (PDF/CSV).
- Advanced admin analytics (occupancy trends, revenue summaries).
- Optional online database support for multi-terminal operations.

Overall, the project establishes a solid baseline and can evolve into a more robust hotel operations platform.

5. Conclusion

This project successfully developed a desktop-based Hotel Management and Booking System using JavaFX, Maven, and SQLite. The system addresses the core operational needs of a small to medium hotel by combining user authentication, role-specific dashboards, booking lifecycle management, and administrative controls within a single application.

The final implementation demonstrates meaningful alignment with the project objectives. Guests can register, book rooms based on date and category, upgrade reservations, and check out. Administrators can monitor bookings and manage room pricing structures. The startup database initialization process improves reliability and allows the application to be used with minimal setup effort.

From an engineering perspective, the project reinforces key development practices: modular architecture, separation of concerns through DAO and controller layers, persistent data management, and user-centric workflow design. These practices contribute to maintainable code and smoother feature expansion.

Although the current version has limitations, especially in areas like security hardening and automated testing, it provides a complete and functional base system. Future enhancements can incrementally transform this prototype into a production-ready platform with stronger security, improved scalability, and richer analytics.

In conclusion, the project meets its academic and functional goals while providing valuable practical insight into building real-world desktop management systems.

References

1. [Oracle Corporation, "Java Platform, Standard Edition Documentation," Oracle Docs](#)
2. [OpenJFX, "JavaFX Documentation," OpenJFX](#)
3. [The Apache Software Foundation, "Maven Documentation," Apache Maven Project](#)
4. [SQLite Consortium, "SQLite Documentation," SQLite](#)
5. [Xerial, "SQLite JDBC Driver," GitHub Repository](#)